

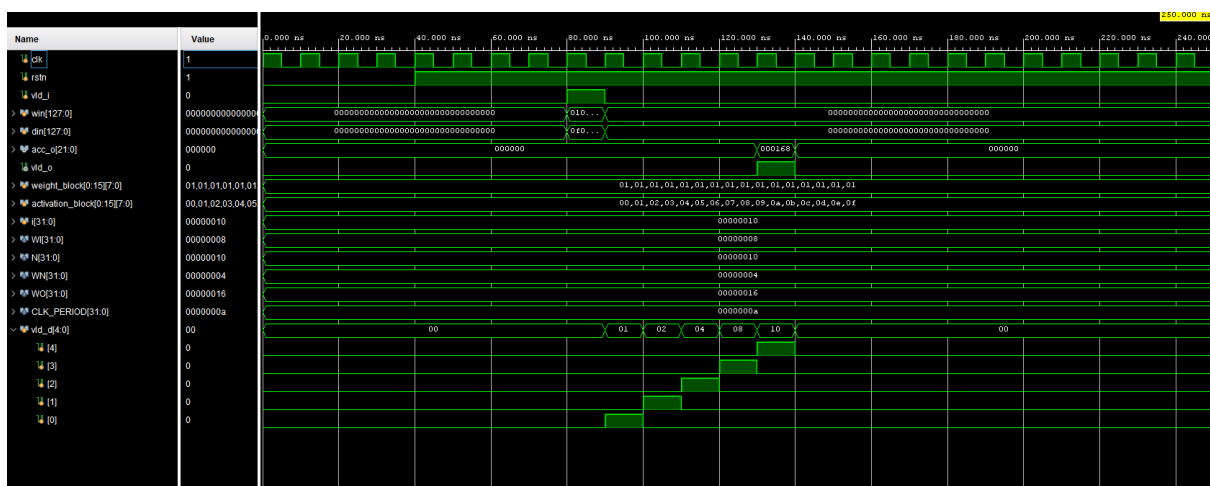
김종원

Day13. LAB1 – MAC (Channel-wise accumulation)

본 실습에서는 CNN Accelerator의 핵심 연산인 MAC(Multiply-Accumulate)을 Verilog로 구현하였다. MAC은 여러 Channel의 Weight와 Activation을 각각 곱한 후 결과를 누적하는 연산으로, 본 실습에서는 총 16개의 Channel을 병렬로 처리하도록 설계하였다.

각 입력 Weight와 Activation은 Register에 저장된 후 16개의 Multiplier를 통해 곱셈이 수행된다. 이후 Hierarchical Adder를 이용하여 $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ 단계로 결과를 합산하고, 최종 결과를 acc_o로 출력하였다. 또한 누락된 Multiplier 2개와 Adder Level 3, Level 4를 추가하여 전체 MAC 구조를 완성하였다.

시뮬레이션 결과는 아래 그림과 같다.



Waveform에서 약 80 ns에서 vld_i가 활성화되며 Weight는 모든 Channel에서 01, Activation은 00부터 0F까지 입력되는 것을 확인할 수 있다. 이후 각 Channel의 곱셈과 Adder Tree 연산을 거쳐 약 130 ns에서 acc_o가 0x168로 출력되며 vld_o가 활성화되었다.

최종 결과가 360이 되는 이유는 Weight가 코드 내부에서 $2 \times \text{win} + 1$ 형태로 변환되기 때문이다. 입력 Weight가 모두 1이므로 실제 연산에 사용되는 Weight는 3이 된다. Activation은 0~15이므로 최종 MAC 결과는

$$3 \times (0 + 1 + \cdots + 15) = 3 \times 120 = 360$$

이다. 시뮬레이션에서 출력된 0x168 역시 Decimal로 변환하면

$$0x168 = 256 + 96 + 8 = 360$$

으로 동일한 결과를 확인할 수 있다.

또한 vld_o는 vld_i보다 5 Clock Cycles 늦게 출력된다. Clock Period가 10 ns이고 vld_i가 약 80 ns, vld_o가 약 130 ns에서 활성화되므로 총 50 ns, 즉 5 cycles의 Latency가 발생한다. 이는 입력 Register와 4단계 Adder Pipeline을 거쳐 최종 결과가 생성되기 때문이다. 따라서 데이터 경로와 timing을 맞추기 위해 vld_i도 Shift Register를 이용하여 5 cycles 지연시켰다.

이번 실습을 통해 16-Channel MAC의 병렬 연산 구조와 Hierarchical Adder를 이용한 누적 과정을 이해하였으며, 시뮬레이션을 통해 최종 결과 360과 5-cycle Pipeline Latency가 정상적으로 동작함을 확인하였다.

Day13. LAB2 – MAC Kernel

본 실습에서는 Problem 1에서 구현한 16-Channel MAC 모듈(mac.v)을 재사용하여 CNN Accelerator의 MAC Kernel을 구현하였다. MAC Kernel은 입력된 Feature Map과 Weight를 이용하여 1×1 Convolution과 3×3 Convolution을 수행하며, MAC에서 출력된 결과를 누적하여 최종 Convolution 결과를 생성한다. 본 실습에서는 누락된 mac_kern.v의 코드를 완성하고, Conv1×1과 Conv3×3 모드에 따른 동작 및 Pipeline Latency를 확인하였다.

시뮬레이션 결과는 아래 그림과 같다.



Waveform을 보면 약 80 ns에서 vld_i가 활성화되며 입력 데이터가 MAC으로 전달된다. Conv1×1 모드에서는 입력값이 순차적으로 처리되며 MAC의 출력이 누적되어 최종 결과가 acc_o에 출력된다. 약 190 ns에서는 acc_o = 0x90이 출력되며, 이는 Decimal 144에 해당한다.

Pipeline이 채워진 이후에는 입력 데이터가 Clock Cycle마다 순차적으로 처리되며, Conv3×3에서는 하나의 Output Pixel을 생성하기 위해 3×3 Window의 9개 위치에 대한

MAC 결과를 순차적으로 누적한다. 따라서 `pix_idx`가 0 → 8까지 증가하면서 9개의 MAC 결과가 순차적으로 처리되고, 이 결과들이 하나의 Output Pixel로 누적되는 것을 확인할 수 있다. 이후 Conv3×3 모드에서는 9개의 MAC 결과를 모두 누적한 뒤 최종 출력을 생성한다. Waveform에서 최종 출력은 `acc_o = 0x2D0`으로 나타나며, 이를 Decimal로 변환하면

$$0x2D0 = 512 + 208 = 720$$

이 된다. 따라서 Conv3×3의 최종 Accumulation 결과는 720임을 확인할 수 있다.

또한 `vld_i`와 `vld_o` 사이의 Pipeline Latency를 확인하면 Conv1×1에서는 6 Clock Cycles, Conv3×3에서는 14 Clock Cycles의 지연이 발생한다.

Conv1×1의 경우 지연은 다음과 같이 구성된다.

- MAC 내부 Pipeline Delay: 5 cycles
- MAC Kernel의 Accumulation: 1 cycle

따라서 전체 Latency는

$$5 + 1 = 6 \text{ cycles}$$

가 된다.

반면 Conv3×3에서는 하나의 출력을 생성하기 위해 9개의 MAC 결과를 모두 누적해야 하므로, Accumulation 과정에 9 Cycle이 추가된다. 따라서 전체 지연은

- MAC 내부 Pipeline Delay: 5 cycles
- MAC Kernel Accumulation: 9 cycles

이며,

$$5 + 9 = 14 \text{ cycles}$$

의 Latency가 발생한다.

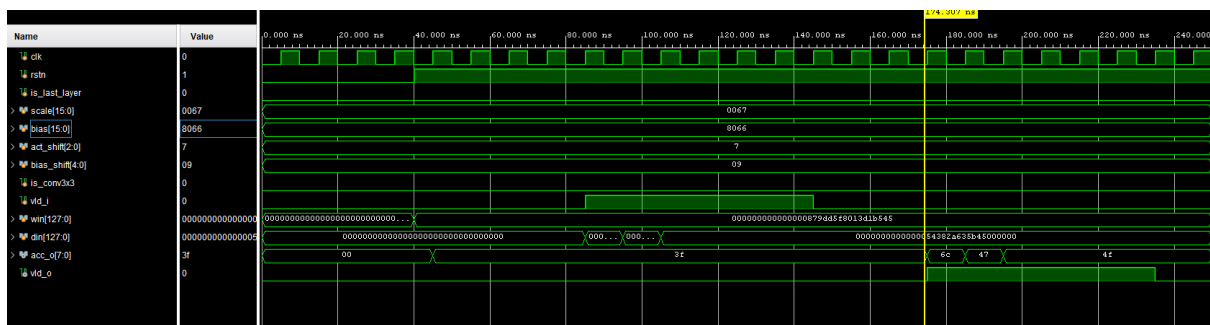
Conv1×1에서는 매 Cycle마다 새로운 Output Pixel을 생성할 수 있지만, Conv3×3에서는 하나의 Output Pixel을 생성하기 위해 9회의 MAC 연산 및 누적 과정이 필요하므로 9 Cycle마다 하나의 Output Pixel이 출력되는 것을 시뮬레이션에서 확인할 수 있었다.

이번 실습을 통해 Conv1×1과 Conv3×3의 동작 방식 및 Pipeline 구조에 따른 Latency 차이를 확인하였다. 또한 시뮬레이션 결과를 통해 Conv1×1에서는 0x90(Decimal 144), Conv3×3에서는 0x2D0(Decimal 720)가 정상적으로 출력되고, 각각 6 Cycle과 14 Cycle의 Pipeline Latency가 발생함을 확인하였다.

Day13. LAB3 – Convolutional Kernel

본 실습에서는 Problem 1과 Problem 2에서 구현한 mac.v와 mac_kern.v를 재사용하여 Convolutional Kernel을 구현하였다. 또한 bias_shifter.v, act_shifter.v 및 conv_kern_tb.v의 누락된 코드를 완성하였다. 특히 conv_kern_tb.v에서는 입력 Feature Map을 순차적으로 입력하여 세 개의 Output Pixel을 계산할 수 있도록 Testbench를 구성하였으며, 이를 통해 Convolution Kernel의 동작을 검증하였다.

시뮬레이션 결과는 아래 그림과 같다.



Waveform을 보면 약 80 ns에서 vld_i가 활성화되며 입력 Feature Map과 Weight가 Convolution Kernel로 전달된다. 이후 MAC 연산과 누적 연산이 수행되고, Batch Normalization과 Activation Quantization을 거쳐 최종 결과가 acc_o로 출력된다. 출력 데이터가 유효한 시점에는 vld_o가 함께 활성화되어 결과가 정상적으로 출력됨을 확인할 수 있다.

Testbench에서는 입력 데이터를 순차적으로 변경하여 세 개의 Output Pixel을 계산하도록 구성하였다. 시뮬레이션 결과 acc_o는 0x6C, 0x47, 0x4F의 값을 순차적으로 출력하였으며, 각 출력은 입력 Feature Map에 대한 Convolution 연산과 Batch Normalization 및 Quantization을 거쳐 생성된 최종 Activation 값이다. 또한 Waveform에서는 초기 출력값 이후 세 개의 Output Pixel에 해당하는 결과가 순차적으로 출력되는 것을 확인할 수 있었다.

시뮬레이션 결과를 통해 vld_o가 활성화되는 동안 acc_o의 값이 정상적으로 출력되는 것을 확인하였으며, 이는 MAC, MAC Kernel, Batch Normalization, Activation Quantization 모듈이 순차적으로 동작하여 최종 Convolution 결과를 생성함을 의미한다.

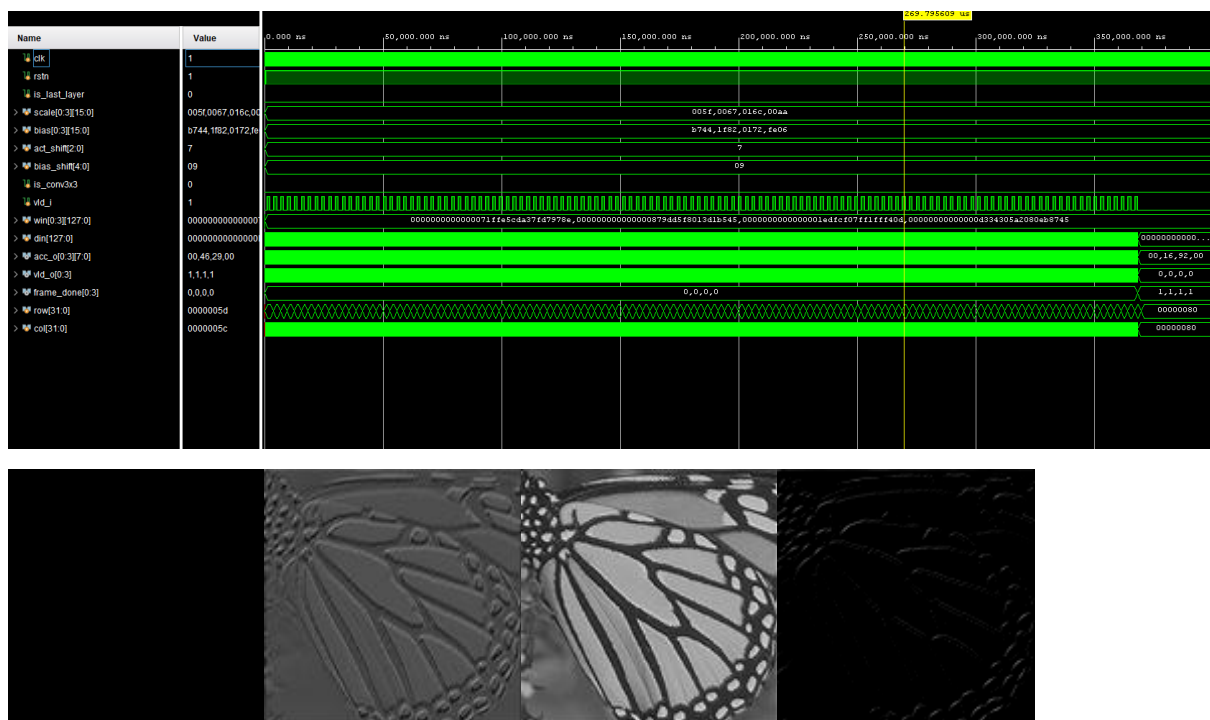
이번 실습을 통해 기존에 구현한 MAC 모듈과 MAC Kernel을 이용하여 Convolutional Kernel을 구성하는 방법을 이해할 수 있었으며, bias_shifter와 act_shifter의 Shift 연산 및 Batch Normalization, Activation Quantization 과정이 정상적으로 수행됨을 확인하였다. 또한 conv_kern_tb.v를 완성하여 세 개의 Output Pixel을 계산하는 Testbench를 구현하

고, 시뮬레이션을 통해 출력값이 정상적으로 생성되는 것을 확인하였으며, Convolution Kernel의 전체 데이터 처리 과정과 Pipeline 기반의 CNN Accelerator 동작을 이해할 수 있었다.

Day14. LAB1 – Sliding window

본 실습에서는 3×3 Sliding Window 알고리즘을 구현하였다. first_layer_tb.v의 누락된 코드를 완성하여 입력 이미지로부터 3×3 Window를 생성하도록 구현하였으며, 이미지의 가장자리에서는 Zero Padding을 적용하여 Convolution 연산이 가능하도록 하였다. 이후 시뮬레이션을 400 μ s 동안 수행하고 Waveform을 확인하였으며, check_hardware_results.m을 이용하여 H/W 시뮬레이션에서 생성된 출력 이미지를 검증하였다.

시뮬레이션 결과는 아래 그림과 같다.



Waveform을 보면 vld_i가 활성화될 때마다 입력 이미지로부터 생성된 3×3 Window가 din으로 전달되는 것을 확인할 수 있다. 이미지 내부에서는 현재 픽셀을 중심으로 주변 9개의 Pixel이 입력되며, 첫 번째 행과 마지막 행, 첫 번째 열과 마지막 열에서는 이미지 밖의 영역을 0으로 채우는 Zero Padding이 적용되어 정상적인 Sliding Window가 생성됨을 확인하였다. 또한 각 Clock Cycle마다 Sliding Window가 한 Pixel씩 이동하면서 Convolution Kernel에 입력되고, 계산된 결과는 vld_o가 활성화되는 시점에 출력 이미지로 저장된다.

시뮬레이션이 완료되면 네 개의 출력 이미지인 convout_layer01_ch01.bmp, convout_layer01_ch02.bmp, convout_layer01_ch03.bmp, convout_layer01_ch04.bmp가 생성된다. 각 출력 이미지는 서로 다른 Weight와 Batch Normalization Parameter(Scale, Bias)를 사용하는 4개의 Convolution Kernel(Channel)의 연산 결과를 나타낸다. 동일한 입력 이미지를 사용하더라도 각 채널의 Filter가 서로 다르므로 출력 이미지의 밝기와 강조되는 특징이 서로 다르게 나타난다.



```
명령 창
MATLAB을 처음 사용한다면 시작하기를 참조하십시오.
Results of the channel 01 are same!
Results of the channel 02 are same!
Results of the channel 03 are same!
Results of the channel 04 are same!
>>
```

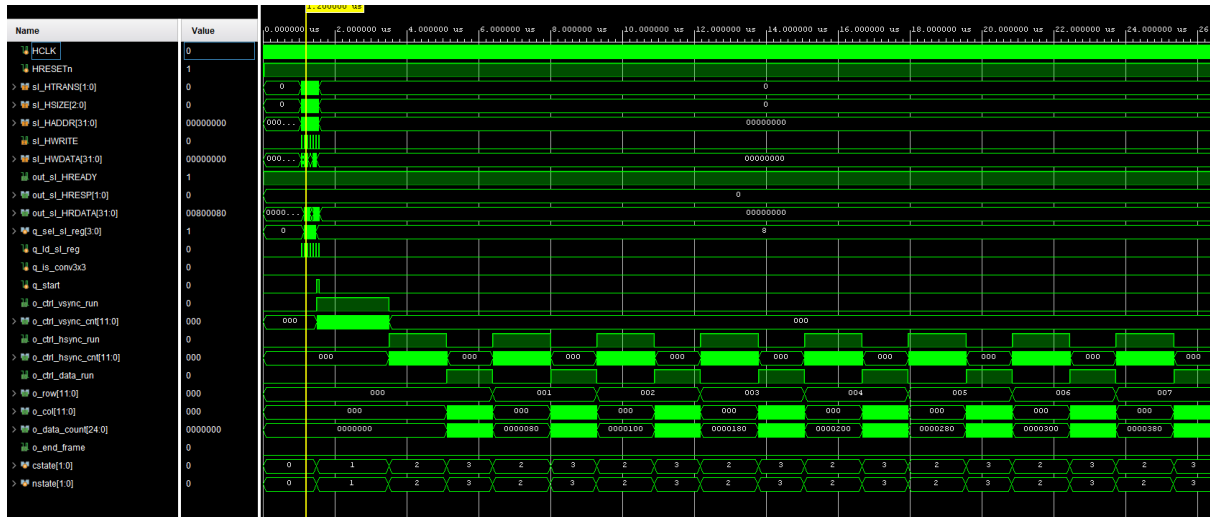
check_hardware_results.m을 이용하여 생성된 네 개의 출력 이미지를 기준 결과와 비교한 결과, 모든 출력 이미지가 정상적으로 생성되었으며 기준 이미지와 일치함을 확인하였다. 이를 통해 Sliding Window 알고리즘과 Convolution Kernel이 정상적으로 동작함을 검증하였다.

이번 실습을 통해 Sliding Window와 Zero Padding을 이용하여 이미지 전체에 대한 Convolution 입력을 생성하는 방법을 이해할 수 있었으며, 서로 다른 Filter를 적용한 여러 개의 Convolution Kernel이 각각의 Feature Map을 생성하는 과정을 확인할 수 있었다.

Day14. LAB2 – AHB interface, CNN Controller

본 실습에서는 AHB Interface와 CNN Controller(FSM)를 구현하였다. cnn_accel.v와 cnn_fsm.v에서 제시된 빈 코드를 완성하여 AHB Slave Interface를 통해 CNN Accelerator를 제어하고, FSM을 이용하여 CNN 연산의 전체 동작을 제어하였다. 또한 400 μ s 동안 시뮬레이션을 수행하여 AHB Interface와 FSM의 상태 전이 및 제어 신호의 동작을 확인하였다.

시뮬레이션 결과는 아래 그림과 같다.

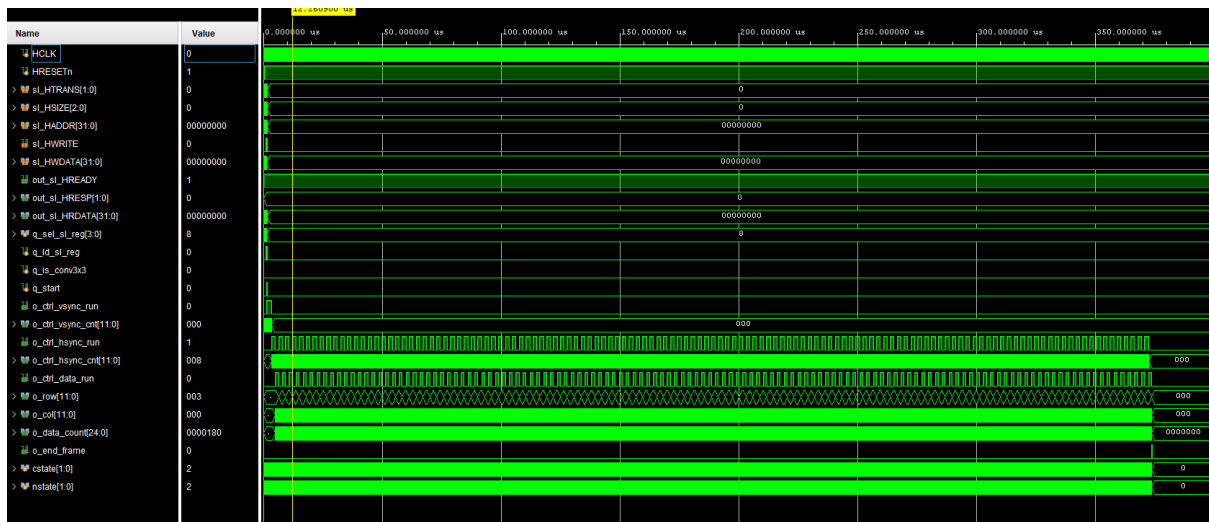
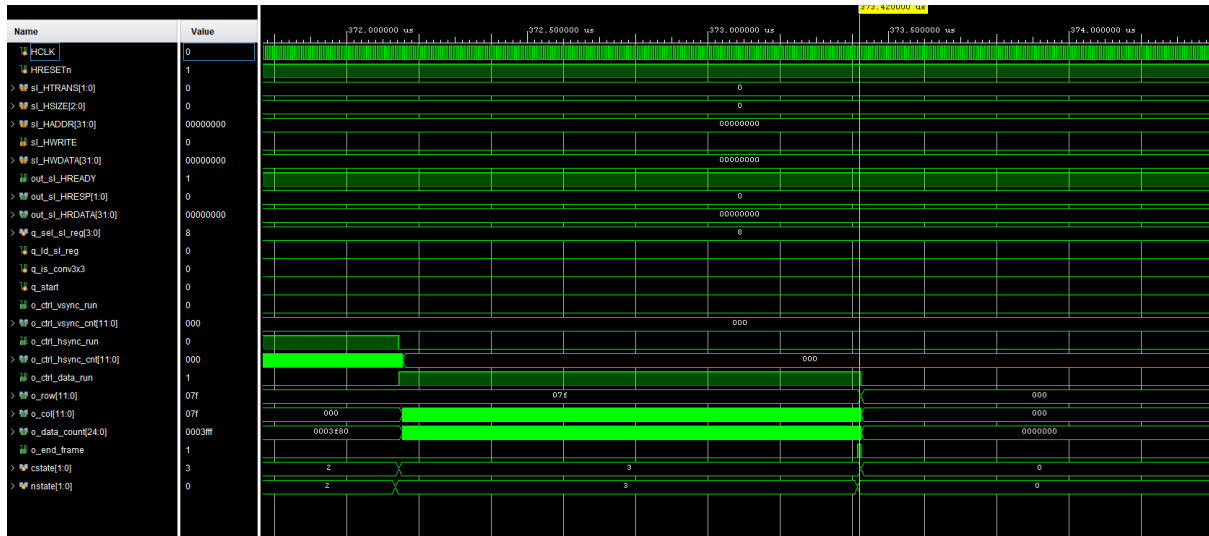


Waveform을 보면 Reset이 해제된 후 AHB Interface를 통해 Layer Configuration과 Control Register가 설정되며, q_start 신호가 활성화되면 CNN Controller가 동작을 시작한다. 이후 FSM은 IDLE → VSYNC → HSYNC → DATA 순서로 상태가 전이된다. Waveform에서 cstate와 nstate를 보면 상태 값이 0 → 1 → 2 → 3으로 변화하는 것을 확인할 수 있으며, 각각 IDLE, VSYNC, HSYNC, DATA 상태를 의미한다.

VSYNC 상태에서는 ctrl_vsync_run이 활성화되어 Start-up Delay를 수행하며, 설정된 Delay가 종료되면 HSYNC 상태로 전환된다. HSYNC 상태에서는 ctrl_hsync_run이 활성화되어 한 Line에 대한 Delay를 수행한 후 DATA 상태로 전환된다.

DATA 상태에서는 ctrl_data_run이 활성화되며 CNN 입력 데이터가 순차적으로 처리된다. 이때 o_row와 o_col이 증가하면서 현재 처리 중인 이미지의 위치를 나타내고, o_data_count는 처리된 Pixel의 개수를 누적한다. Waveform에서도 o_col이 0에서 127까지 증가한 후 다시 0으로 초기화되며, 동시에 o_row가 1씩 증가하는 것을 확인할 수 있다. 또한 o_data_count 역시 데이터가 처리될 때마다 증가하여 전체 Frame의 진행 상황을 나타낸다.

시뮬레이션의 마지막 부분 및 전체는 아래 그림들과 같다.



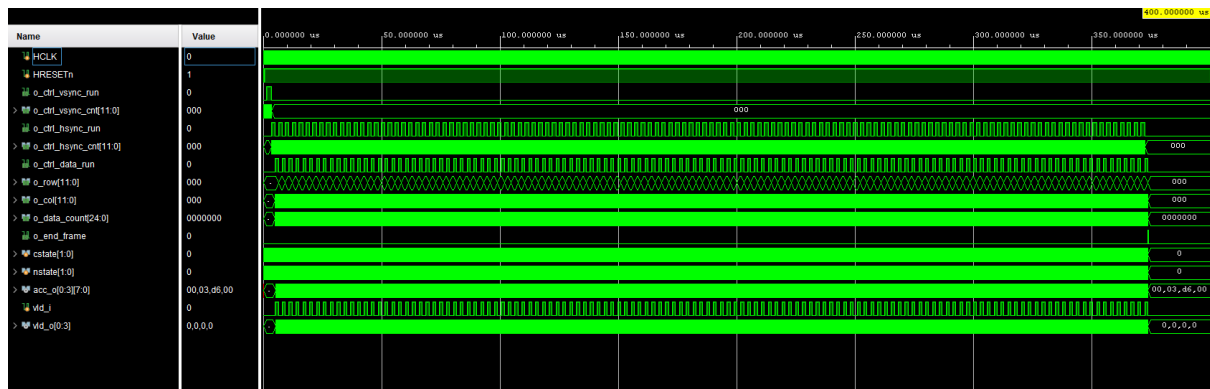
마지막 Pixel까지 처리되면 o_end_frame 신호가 High가 되며 Frame 처리가 완료된다. 동시에 ctrl_data_run이 비활성화되고 o_row, o_col, o_data_count가 초기값으로 복귀한다. FSM 역시 DATA 상태에서 IDLE 상태로 전이하며 다음 Layer의 연산을 대기하는 것을 확인하였다.

이번 실습에서는 cnn_accel.v에서 AHB Register Interface와 Layer Configuration Register를 구현하고, cnn_fsm.v에서 CNN Accelerator를 제어하는 FSM을 구현하였다. 시뮬레이션 결과를 통해 AHB Interface를 이용한 Register 설정과 FSM의 상태 전이, VSYNC 및 HSYNC Delay 제어, DATA Processing, 그리고 Frame 종료 과정이 모두 정상적으로 동작함을 확인하였다. 이를 통해 AHB Interface와 FSM이 CNN Accelerator의 전체 데이터 처리 흐름과 연산 순서를 제어하는 핵심 요소임을 이해할 수 있었다.

Day14. LAB3 – Simplified CNN accelerator IP

본 실습에서는 Simplified CNN Accelerator IP를 구현하였다. Problem 5에서 구현한 CNN FSM(cnn_fsm.v)을 재사용하고, cnn_accel.v의 빈 코드를 완성하여 AHB Interface를 통한 Configuration Register를 구성하였다. 또한 첫 번째 Convolution Layer에 대한 3×3 Sliding Window를 생성하기 위해 입력 데이터(din)와 입력 유효 신호(vld_i)를 구현하고, Output Channel 3의 Filter를 추가하였다. 마지막으로 400 μs 동안 시뮬레이션을 수행하여 CNN Accelerator의 전체 동작과 Convolution 결과를 확인하였다.

시뮬레이션 결과는 아래 그림과 같다.



Waveform을 보면 Reset이 해제된 후 CNN Controller가 동작을 시작하며, Problem 5에서 구현한 FSM이 IDLE → VSYNC → HSYNC → DATA 순서로 상태를 전이하는 것을 확인할 수 있다. VSYNC 구간에서는 Start-up Delay가 수행되고, HSYNC 구간에서는 각 이미지 Line에 대한 Delay가 수행된다. 이후 DATA 상태에서 ctrl_data_run 신호가 활성화되면서 입력 이미지가 순차적으로 처리된다.

DATA 구간에서는 FSM에서 생성한 row와 col 값을 이용하여 3×3 Sliding Window가 생성된다. 입력 유효 신호인 vld_i는 ctrl_data_run과 함께 활성화되어 Convolution Kernel에 입력 데이터가 전달된다. 또한 이미지의 가장자리에서는 Boundary Checking을 통해 Zero Padding이 적용되도록 구현하였다. 따라서 첫 번째 행, 마지막 행, 첫 번째 열, 마지막 열에서는 존재하지 않는 Pixel 대신 0이 입력되며, 내부 Pixel에서는 주변 9개의 Pixel이 Sliding Window 형태로 din에 저장된다.

또한 4개의 Convolution Kernel이 동시에 동작하면서 각각의 Filter를 이용하여 입력 데이터를 처리한다. 각 Kernel의 출력은 acc_o[0]~acc_o[3]로 생성되며, 이를 하나의 32-bit 출력인 out_pixel로 결합하였다. Waveform에서도 DATA 구간 동안 vld_i가 활성화될 때마다 Convolution 결과가 생성되고, 출력 Valid 신호인 out_valid(vld_o) 역시 함께 활성화되는 것을 확인할 수 있다.



```

명령 창
MATLAB을 처음 사용한다면 시작하기를 참조하십시오.
Results of the channel 01 are same!
Results of the channel 02 are same!
Results of the channel 03 are same!
Results of the channel 04 are same!
>>

```

마지막 Pixel까지 처리되면 o_end_frame 신호가 활성화되며 하나의 Frame 처리가 완료된다. 이후 ctrl_data_run이 비활성화되고 row, col, data_count가 모두 초기값으로 복귀한다. FSM 역시 DATA 상태에서 다시 IDLE 상태로 전이하여 다음 연산 또는 다음 Frame의 처리를 대기하는 것을 확인하였다. 또한 4개의 Output Channel에서 생성된 Convolution 결과가 정상적으로 출력되었으며, MATLAB의 check_hardware_results.m을 이용하여 생성된 출력 이미지(convout_ch01~convout_ch04)을 확인한 결과 하드웨어 결과가 정상적으로 생성됨을 확인하였다.

이번 실습에서는 Problem 5에서 구현한 FSM을 재사용하여 Simplified CNN Accelerator IP를 완성하고, AHB Interface를 통해 CNN Accelerator의 Configuration Register를 구성하였다. 또한 Sliding Window와 Zero Padding을 이용하여 첫 번째 Convolution Layer의 입력을 생성하고, 4개의 Convolution Kernel이 병렬로 동작하도록 구현하였다. 시뮬레이션 결과를 통해 FSM의 상태 전이, Sliding Window 생성, 입력 데이터 전달, Convolution 연산 및 출력 생성 과정이 모두 정상적으로 수행됨을 확인하였다. 이를 통해 CNN Accelerator에서 FSM과 Sliding Window가 입력 데이터를 효율적으로 공급하고, 다중 Convolution Kernel이 병렬 연산을 수행하여 CNN의 첫 번째 Layer를 구현하는 핵심 요소임을 이해할 수 있었다.

Day15. LAB1 – weight, bias, and scale buffers

본 실습에서는 이전에 구현한 cnn_fsm.v를 재사용하여 CNN Accelerator를 완성하였다. 또한 cnn_accel.v의 누락된 코드를 구현하여 Configuration Register를 설정하고, Sliding Window를 위한 din 및 vld_i 생성 회로를 추가하였다. 마지막으로 Weight, Scale, Bias를 저장하는 Buffer를 구현하여 Convolution Kernel에 필요한 파라미터를 공급하도록 하였다.

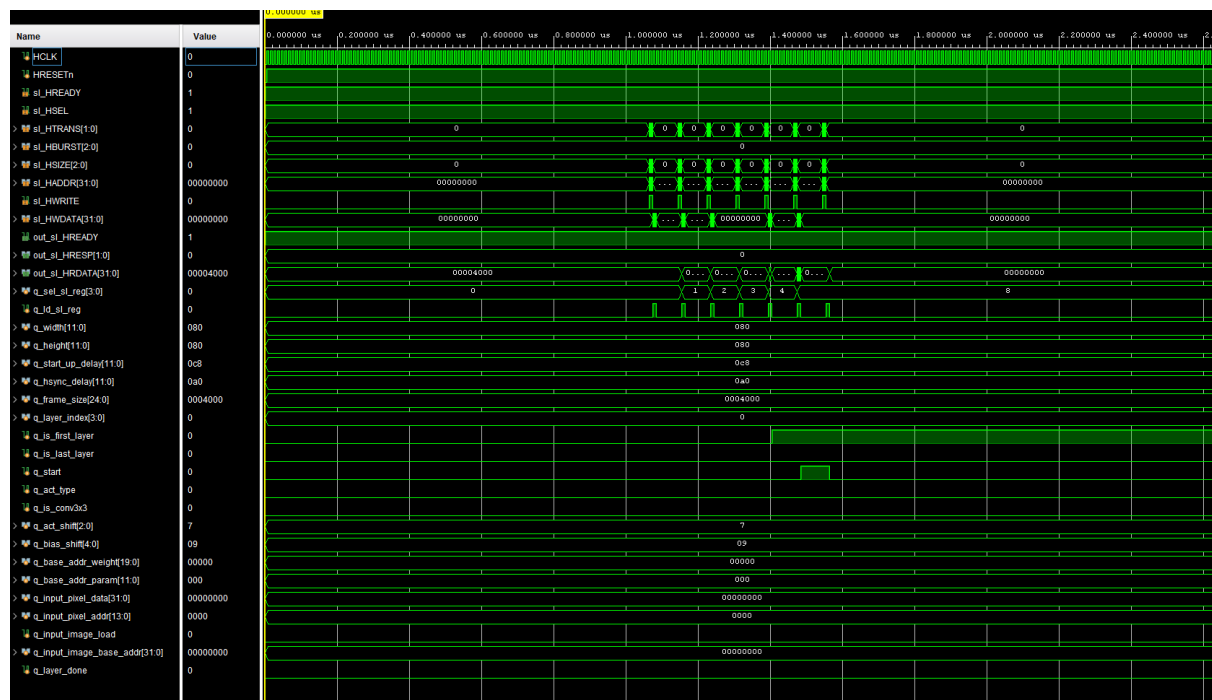
Configuration Register에서는 Layer의 종류와 Convolution 방식(1×1 또는 3×3), Activation Shift, Bias Shift 등의 파라미터를 설정할 수 있도록 구현하였다. 또한 Layer

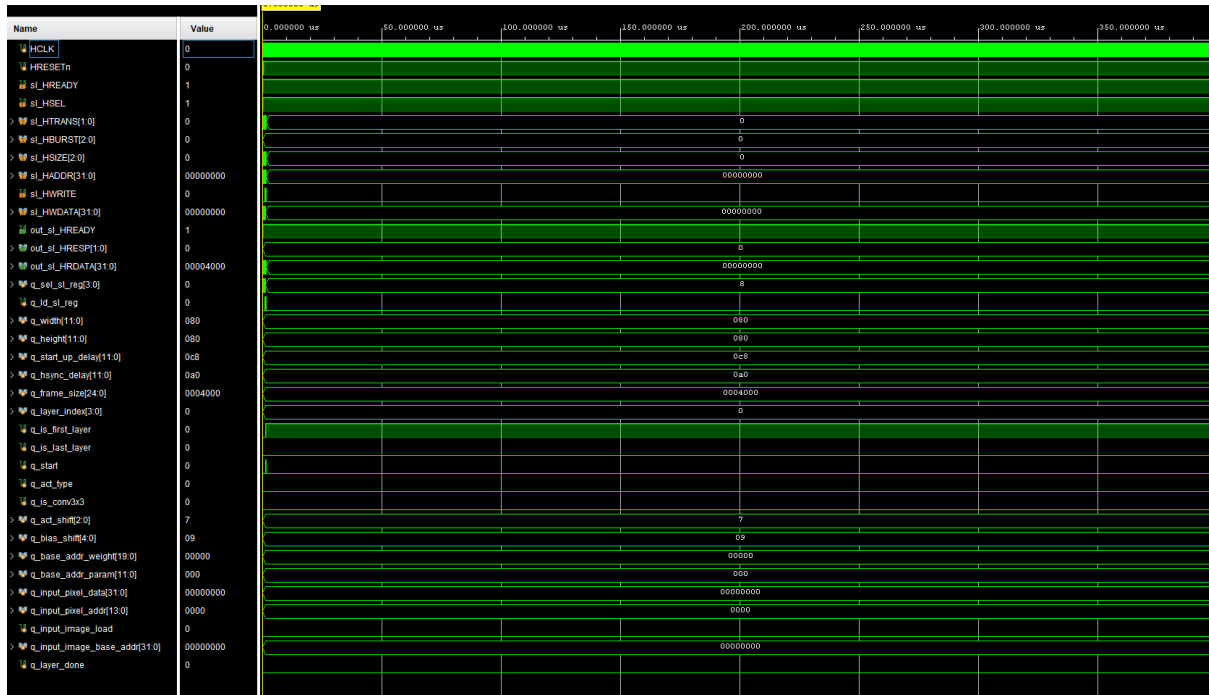
Index를 이용하여 여러 Layer를 지원할 수 있는 구조를 유지하였으며, 제공된 Convolution Layer의 Weight, Scale, Bias Parameter를 Buffer에 저장하여 사용하였다.

Weight Buffer와 Parameter(Buffer for Scale/Bias)는 SPRAM(Single-Port RAM)을 이용하여 구현하였다. VSYNC 구간에서 Buffer Enable 신호를 활성화하여 필요한 Weight와 Batch Normalization Parameter를 읽어오도록 설계하였다. 읽어온 Weight는 각 Convolution Kernel의 win 입력으로 전달되고, Scale과 Bias는 Batch Normalization 모듈로 전달되어 연산에 사용된다.

입력 이미지는 Sliding Window 구조를 이용하여 생성하였다. DATA 상태에서 FSM이 생성하는 row와 col 값을 이용하여 현재 Pixel 주변의 3×3 Window를 구성하였으며, 이미지의 가장자리에서는 Boundary Checking을 통해 Zero Padding을 적용하였다. 생성된 Window는 din으로 전달되고, DATA 구간 동안 vld_i가 활성화되어 Convolution Kernel이 연속적으로 입력 데이터를 처리하도록 구현하였다.

시뮬레이션은 400 μs 동안 수행하였으며, 그 결과는 아래 그림과 같다.





명령 창

MATLAB을 처음 사용한다면 [시작하기](#)를 참조하십시오.

```
Results of the channel 01 are same!
Results of the channel 02 are same!
Results of the channel 03 are same!
Results of the channel 04 are same!
>>
```

Waveform을 보면 Reset 이후 AHB Interface를 통해 Configuration Register가 설정되는 것을 확인할 수 있다. 이후 Layer Configuration Register가 설정되면서 q_is_first_layer, q_layer_index, q_act_shift, q_bias_shift 등의 값이 저장되고, q_start 신호가 활성화되면서 CNN Accelerator가 동작을 시작한다. 또한 Weight Buffer와 Parameter Buffer의 Enable 신호가 활성화되어 Weight, Scale, Bias를 순차적으로 읽어오는 것을 확인할 수 있으며, 이후 DATA 구간에서 din과 vld_i가 생성되어 Convolution Kernel로 입력 데이터가 전달된다.

시뮬레이션이 완료된 후 check_hardware_results.m을 이용하여 Hardware 시뮬레이션에서 생성된 출력 이미지를 검증하였다. 생성된 출력 이미지는 기준 결과와 동일하게 나타났다으며, 이를 통해 Weight Buffer, Scale/Bias Buffer, Sliding Window 및 Convolution Kernel이 모두 정상적으로 동작함을 확인하였다.

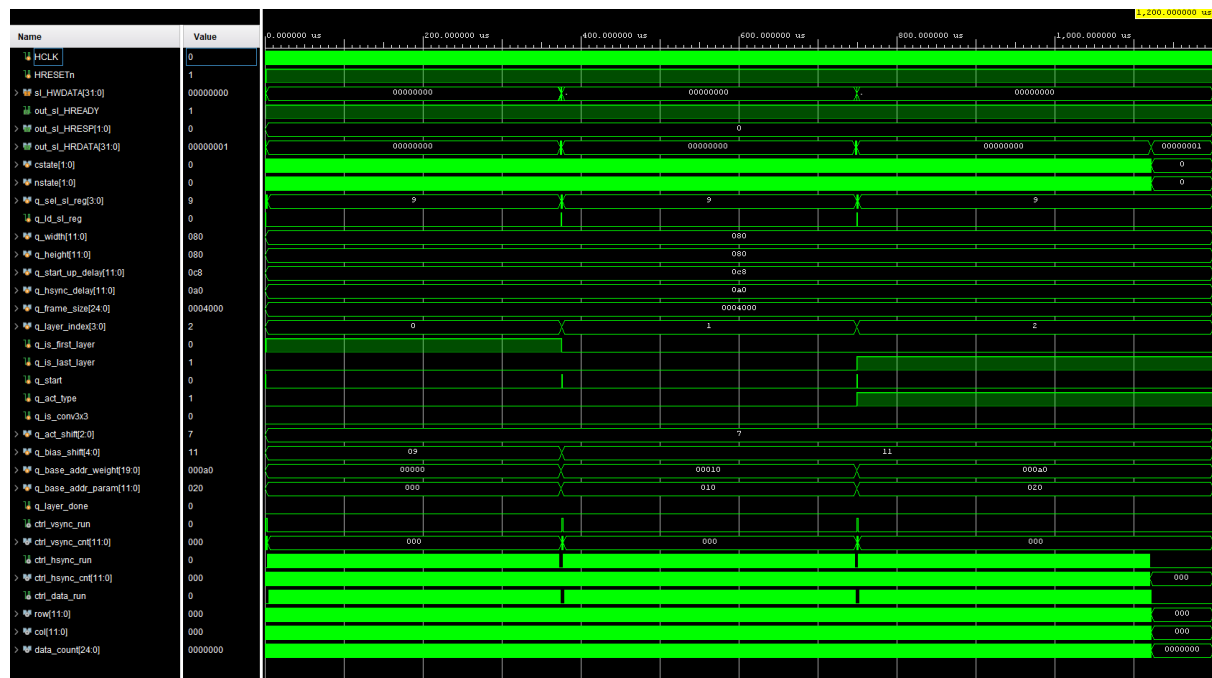
Day15. LAB2 – Feature map buffers

본 실습에서는 Weight, Scale, Bias를 저장하는 Buffer를 구현하고, 각 Layer에 해당하는 Parameter를 선택하여 총 3개의 Convolution Layer를 순차적으로 실행하도록 설계하였다.

Weight Buffer와 Parameter Buffer(Scale/Bias)는 SPRAM(Single-Port RAM)을 이용하여 구현하였다. VSYNC 구간에서 Buffer Enable 신호를 활성화하여 각 Layer에 필요한 Weight와 Batch Normalization Parameter를 읽어오도록 설계하였다. 읽어온 Weight는 Convolution Kernel의 입력으로 전달되며, Scale과 Bias는 Batch Normalization 연산에 사용된다. Layer가 변경될 때마다 Base Address가 변경되어 서로 다른 Weight와 Parameter를 사용하는 것을 확인할 수 있었다.

입력 이미지는 Sliding Window 구조를 이용하여 생성하였다. DATA 상태에서 FSM이 생성하는 row와 col 값을 이용하여 현재 Pixel 주변의 3×3 Window를 구성하였으며, 이미지의 가장자리에서는 Boundary Checking을 통해 Zero Padding을 적용하였다. 생성된 Window는 din으로 전달되고, DATA 구간 동안 vld_i가 활성화되어 Convolution Kernel이 연속적으로 입력 데이터를 처리하도록 구현하였다.

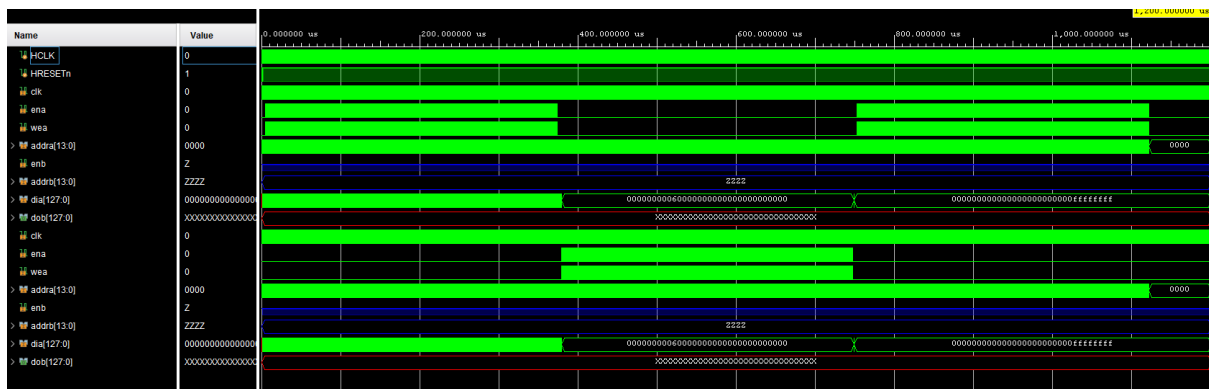
시뮬레이션은 약 1200 μ s 동안 수행하였으며, 그 결과는 아래 그림과 같다.





```
명령 창
MATLAB을 처음 사용한다면 시작하기를 참조하십시오.
Results of the channel 01 are same!
Results of the channel 02 are same!
Results of the channel 03 are same!
Results of the channel 04 are same!
>>
```

Waveform을 보면 Reset 이후 AHB Interface를 통해 Configuration Register가 정상적으로 설정되는 것을 확인할 수 있다. 이후 Layer Configuration Register가 설정되면서 q_layer_index, q_act_shift, q_bias_shift, q_base_addr_weight, q_base_addr_param 등의 값이 Layer에 따라 변경되며, q_start 신호가 활성화되어 CNN Accelerator의 연산이 시작된다. 파형에서는 q_layer_index가 0 → 1 → 2로 증가하면서 각 Layer가 순차적으로 실행되는 것을 확인할 수 있으며, 이에 따라 Weight와 Parameter의 Base Address도 각각 0x00000 → 0x00010 → 0x000A0, 0x000 → 0x010 → 0x020으로 변경되어 각 Layer에 필요한 Weight와 Scale/Bias가 정상적으로 선택되는 것을 확인하였다. 또한 Weight Buffer와 Parameter Buffer의 Enable 신호가 활성화되어 필요한 데이터를 순차적으로 읽어오고, DATA 구간에서는 din과 vld_i가 생성되어 Convolution Kernel로 입력 데이터가 전달되는 것을 확인하였다.



또한 위 그림은 CNN Accelerator에서 사용하는 Dual Buffer(Ping-Pong Buffer)의 시뮬레이션 결과를 나타낸 것이다. 두 개의 Feature Map Buffer를 동일한 구조로 구현하고, 각 Buffer에 대해 ena, wea, addra, dia 등의 신호를 사용하여 Output Feature Map을 저장하도록 구성하였다. Waveform에서는 Layer의 진행에 따라 두 Buffer의 Enable 및 Write Enable 신호가 서로 번갈아 활성화되는 모습을 확인할 수 있으며, 이를 통해 Layer가 변경될 때마다 Feature Map 저장에 사용되는 Buffer가 교대로 선택되는 것을 확인할 수 있다.

CNN Accelerator에서는 두 개의 Feature Map Buffer를 번갈아 사용하여 서로 다른 Layer의 Feature Map을 저장할 수 있도록 Ping-Pong Buffer 구조를 사용한다. 한 Layer의 연산 결과는 현재 선택된 Buffer에 저장되며, 다음 Layer에서는 다른 Buffer를 선택하여 새로운 Feature Map을 저장하는 방식으로 Buffer의 역할을 교대로 변경한다. 이를 통해 각 Layer의 Output Feature Map을 독립적으로 관리할 수 있으며, Layer가 증가함에 따라 Buffer A와 Buffer B를 반복적으로 사용하는 구조를 구현할 수 있다.

Dual Buffer의 동작은 다음과 같이 구성된다.

- Layer 0: 입력 영상을 처리하여 Buffer A에 Feature Map을 저장한다.
- Layer 1: Buffer B를 선택하여 새로운 Layer의 Output Feature Map을 저장한다.
- Layer 2: 다시 Buffer A를 선택하여 다음 Output Feature Map을 저장한다.

이와 같이 두 Buffer는 Layer가 변경될 때마다 저장 역할을 서로 교환(Ping-Pong)하며 동작한다. 이러한 구조를 사용하면 하나의 Buffer에 모든 Layer의 Feature Map을 계속 저장하는 대신 두 개의 Buffer를 교대로 사용할 수 있어 Layer 간 Feature Map을 효율적으로 관리할 수 있다. 또한 Buffer 선택 신호를 이용하여 현재 Layer의 Output을 어느 Buffer에 저장할지 제어할 수 있으므로 CNN Accelerator의 Feature Map 관리 구조를 단순화할 수 있다.

시뮬레이션이 완료된 후 `check_hardware_results.m`을 이용하여 Hardware 시뮬레이션에서 생성된 출력 이미지를 검증하였다. 생성된 출력 이미지는 기준 결과와 동일하게 나타났다. 이를 통해 Configuration Register, Weight Buffer, Scale/Bias Buffer, Sliding Window, Dual Buffer 및 Convolution Kernel이 모두 정상적으로 동작함을 확인하였다.

Day15. LAB3 – Finite State Machine

본 실습에서는 이전 단계에서 구현한 CNN Accelerator를 기반으로 Conv1x1과 Conv3x3 연산을 모두 지원하도록 CNN Accelerator를 확장하였다. `cnn_fsm.v`에서는 기존 FSM 구조를 재사용하고 Conv3x3 연산을 위한 `pix_idx` 제어를 추가하였으며, `cnn_accel.v`에서는 기존 Configuration Register와 Sliding Window 구조를 재사용하고 Weight, Scale, Bias Buffer를 추가하여 각 Layer에 필요한 Parameter를 공급하도록 구현하였다.

먼저 `cnn_fsm.v`에서는 ST_IDLE, ST_VSYNC, ST_HSYNC, ST_DATA의 4개 상태를 기반으로 CNN Accelerator의 동작 순서를 제어하도록 구성하였다. `q_start`가 활성화되면 VSYNC와 HSYNC Delay를 거쳐 DATA 상태로 진입하며, DATA 상태에서는 `row`, `col`, `data_count`를 이용하여 현재 처리 중인 Pixel의 위치와 Frame 내 데이터 위치를 관리하도록 구현하였다. 또한 `col`이 `q_width-1`에 도달하면 한 Line의 처리가 종료되어 다시 HSYNC 상태로

이동하고, data_count가 Frame 마지막 위치에 도달하면 end_frame을 통해 해당 Frame의 처리를 종료하도록 구성하였다.

Conv3x3 연산을 지원하기 위해 q_is_conv3x3 신호를 이용하여 Convolution Mode를 구분하였으며, Conv3x3일 경우 pix_idx가 0부터 8까지 증가하도록 구현하였다. 이를 통해 하나의 출력 Pixel을 계산하기 위해 3×3 Kernel의 9개 위치를 순차적으로 처리할 수 있도록 하였다. Waveform에서도 q_is_conv3x3가 1로 유지되는 것을 확인할 수 있으며, pix_idx가 0에서 시작하여 반복적으로 변화하면서 Conv3x3 연산이 수행되는 것을 확인할 수 있다.

cnn_accel.v에서는 Problem 8에서 사용한 Configuration Register 설정 방식과 입력 데이터 생성 구조를 그대로 재사용하였다. AHB Interface를 통해 q_layer_index, q_is_last_layer, q_is_conv3x3, q_act_shift, q_bias_shift 등의 Layer Configuration을 설정하고, 각 Layer에 따라 Weight와 Parameter의 Base Address를 변경하도록 구성하였다. Waveform을 보면 q_layer_index가 0 → 1 → 2로 순차적으로 변경되며, 이에 따라 q_base_addr_weight도 0x000000 → 0x000010 → 0x000A0, q_base_addr_param도 0x000 → 0x010 → 0x020으로 변경되는 것을 확인할 수 있다. 이를 통해 각 Layer에서 서로 다른 Weight와 Scale/Bias Parameter가 선택되어 사용되는 것을 확인하였다.

또한 Weight, Scale, Bias를 저장하기 위한 Buffer를 SPRAM 기반으로 구현하였다. VSYNC 구간에서 ctrl_vsync_run과 함께 Buffer Enable 신호를 활성화하고 Address를 증가시키면서 각 Layer에 필요한 Weight와 Parameter를 순차적으로 읽어오도록 구성하였다. Waveform에서도 Layer가 변경될 때마다 q_base_addr_weight와 q_base_addr_param이 변경되고, 이에 따라 win, scale, bias 값이 변경되는 모습을 확인할 수 있다. 이를 통해 각 Layer의 Convolution 연산에 필요한 Parameter가 정상적으로 Buffer에서 읽혀 Convolution Kernel로 전달되는 것을 확인하였다.

입력 이미지는 기존 Sliding Window 구조를 재사용하여 생성하였다. FSM에서 생성되는 row와 col을 이용하여 현재 Pixel 주변의 3×3 Window를 구성하고, Image Boundary에서는 Boundary Checking을 통해 Zero Padding을 적용하였다. 생성된 Window는 din으로 전달되며 DATA 구간에서 vld_i가 활성화되어 Convolution Kernel이 입력 데이터를 연속적으로 처리하도록 설계하였다. Waveform에서도 DATA 처리 구간에서 din과 vld_i가 반복적으로 발생하며, 입력 Window 데이터가 Convolution Kernel로 전달되는 것을 확인할 수 있다.

시뮬레이션은 4,000 μ s 동안 수행하였으며, 그 결과는 아래 그림들과 같다.



```

명령 창
MATLAB을 처음 사용한다면 시작하기를 참조하십시오.

Results of the channel 01 are same!
Results of the channel 02 are same!
Results of the channel 03 are same!
Results of the channel 04 are same!
>>

```

Waveform을 보면 Reset 이후 CNN Accelerator의 Configuration Register가 설정되고, 이후 각 Layer가 순차적으로 실행되는 것을 확인할 수 있다. 처음에는 q_layer_index가 0으로 설정되어 있으며 이후 1, 2로 변경되면서 각 Layer의 연산이 순차적으로 수행된다. 또한 q_start가 각 Layer의 시작 시점에서 활성화되고, Layer 연산이 종료된 후 다음 Layer의 Configuration으로 변경되는 모습을 확인할 수 있다. q_is_last_layer 역시 마지막 Layer에서 활성화되어 Layer의 종료 여부를 구분하는 것을 확인할 수 있다.

특히 Waveform 하단의 win, din, vld_i, acc_o, vld_o, scale, bias 신호를 통해 실제 CNN 연산 데이터 경로가 동작하는 것을 확인할 수 있다. win에는 Layer에 해당하는 Weight 값이 로드되고, scale과 bias 역시 Layer 변경에 따라 새로운 값으로 변경된다. DATA 구간에서는 din과 vld_i가 활성화되며 입력 데이터가 Convolution Kernel로 전달되고, 연산 결과로 acc_o와 vld_o가 발생하는 것을 확인할 수 있다. 이를 통해 Configuration 설정부

터 Parameter Loading, Input Window 생성 및 Convolution 연산 결과 생성까지 전체 데이터 경로가 정상적으로 동작함을 확인하였다.

또한 frame_done[0:3] 신호를 통해 각 Output Channel의 Frame 연산이 완료되는 과정도 확인할 수 있으며, 전체 시뮬레이션 동안 각 Layer의 연산이 순차적으로 진행되는 것을 확인할 수 있다.

시뮬레이션이 완료된 후 check_hardware_results.m을 이용하여 Hardware 시뮬레이션에서 생성된 Output Image를 검증하였다. 생성된 결과를 기준 결과와 비교하여 Convolution 연산 결과가 정상적으로 생성되는 것을 확인하였으며, cnn_fsm.v의 FSM 제어, Conv3x3의 pix_idx 제어, Weight/Scale/Bias Buffer, Sliding Window 및 Convolution Kernel이 정상적으로 연동되어 동작하였다. 이 과정들을 통해 Conv1x1 기반 CNN Accelerator를 Conv3x3까지 확장하면서 FSM과 pix_idx를 이용해 3x3 연산의 순차적인 데이터 제어가 이루어지는 구조를 이해하였다.